ANSI SQL Using MySQL — Exercises

1. User Upcoming Events

Task: Show a list of all upcoming events a user is registered for in their city, sorted by date. (Example shown for user_id = 1, Alice Johnson.)

SQL Query

```
SELECT e.event_id, e.title, e.city, e.start_date
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
WHERE r.user_id = 1
  AND e.city = (SELECT city FROM Users WHERE user_id = 1)
  AND e.status = 'upcoming'
ORDER BY e.start_date;
```

Result

event_id	title	city	start_date
1	Tech Innovators Meetup	New York	2025-06-10 10:00:00

---

2. Top Rated Events

Task: Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.

SQL Query

```
SELECT e.event_id, e.title, ROUND(AVG(f.rating),2) AS avg_rating, COUNT(f.feedback_id) AS
num_feedback
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;
```

Result

event_id	title	avg_rating	num_feedback

---

3. Inactive Users

Task: Retrieve users who have not registered for any events in the last 90 days.

SQL Query

```
SELECT u.user_id, u.full_name, u.email
FROM Users u
WHERE u.user_id NOT IN (
  SELECT r.user_id FROM Registrations r
  WHERE r.registration_date >= DATE_SUB(CURDATE(), INTERVAL 90 DAY)
```

```
)
ORDER BY u.user_id;
```

Result

user_id	full_name	email

---

4. Peak Session Hours

Task: Count how many sessions are scheduled between 10 AM to 12 PM for each event.

SQL Query

```
SELECT e.event_id, e.title, COUNT(s.session_id) AS session_count
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
  AND TIME(s.start_time) >= '10:00:00' AND TIME(s.start_time) < '12:00:00'
GROUP BY e.event_id, e.title
ORDER BY e.event_id;
```

Result

event_id	title	session_count
1	Tech Innovators Meetup	2
2	AI & ML Conference	0
3	Frontend Development Bootcamp	1

---

5. Most Active Cities

Task: List the top 5 cities with the highest number of distinct user registrations.

SQL Query

```
SELECT u.city, COUNT(DISTINCT r.user_id) AS distinct_users
FROM Registrations r
JOIN Users u ON r.user_id = u.user_id
GROUP BY u.city
ORDER BY distinct_users DESC
LIMIT 5;
```

Result

city	distinct_users
New York	2
Los Angeles	2
Chicago	1

---

6. Event Resource Summary

Task: Generate a report showing the number of resources (PDFs, images, links) uploaded for each event.

SQL Query

```
SELECT e.event_id, e.title,
  SUM(CASE WHEN r.resource_type='pdf' THEN 1 ELSE 0 END) AS pdf_count,
  SUM(CASE WHEN r.resource_type='image' THEN 1 ELSE 0 END) AS image_count,
  SUM(CASE WHEN r.resource_type='link' THEN 1 ELSE 0 END) AS link_count,
  COUNT(r.resource_id) AS total_resources
FROM Events e
LEFT JOIN Resources r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY e.event_id;
```

Result

event_id	title	pdf_count	image_count	link_count	total_resources
1	Tech Innovators Meetup	1	0	0	1
2	AI & ML Conference	0	1	0	1
3	Frontend Development Bootcamp	0	0	1	1

---

7. Low Feedback Alerts

Task: List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

SQL Query

```
SELECT u.full_name, f.rating, f.comments, e.title AS event_name
FROM Feedback f
JOIN Users u ON f.user_id = u.user_id
JOIN Events e ON f.event_id = e.event_id
WHERE f.rating < 3;
```

Result

full_name	rating	comments	event_name

---

8. Sessions per Upcoming Event

Task: Display all upcoming events with the count of sessions scheduled for them.

SQL Query

```
SELECT e.event_id, e.title, COUNT(s.session_id) AS session_count
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title
ORDER BY e.event_id;
```

Result

event_id	title	session_count
1	Tech Innovators Meetup	2
3	Frontend Development Bootcamp	1

---

9. Organizer Event Summary

Task: For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

SQL Query

```
SELECT u.user_id, u.full_name,
  COUNT(e.event_id) AS total_events,
  SUM(CASE WHEN e.status='upcoming' THEN 1 ELSE 0 END) AS upcoming_count,
  SUM(CASE WHEN e.status='completed' THEN 1 ELSE 0 END) AS completed_count,
  SUM(CASE WHEN e.status='cancelled' THEN 1 ELSE 0 END) AS cancelled_count
FROM Users u
JOIN Events e ON u.user_id = e.organizer_id
GROUP BY u.user_id, u.full_name
ORDER BY total_events DESC;
```

Result

user_id	full_name	total_events	upcoming_count	completed_count	cancelled_count
1	Alice Johnson	1	1	0	0
2	Bob Smith	1	1	0	0
3	Charlie Lee	1	0	1	0

---

10. Feedback Gap

Task: Identify events that had registrations but received no feedback at all.

SQL Query

```
SELECT e.event_id, e.title
FROM Events e
WHERE e.event_id IN (SELECT event_id FROM Registrations)
  AND e.event_id NOT IN (SELECT event_id FROM Feedback);
```

Result

event_id	title
3	Frontend Development Bootcamp

---

11. Daily New User Count

Task: Find the number of users who registered each day in the last 7 days.

SQL Query

```
WITH RECURSIVE date_range AS (
  SELECT CURDATE() - INTERVAL 6 DAY AS dt
  UNION ALL
  SELECT dt + INTERVAL 1 DAY FROM date_range WHERE dt < CURDATE()
)
SELECT dr.dt AS registration_day, COUNT(u.user_id) AS new_users
FROM date_range dr
LEFT JOIN Users u ON DATE(u.registration_date) = dr.dt
GROUP BY dr.dt
ORDER BY dr.dt;
```

Result

registration_day	new_users
2025-06-26	0
2025-06-27	0
2025-06-28	0
2025-06-29	0
2025-06-30	0
2025-07-01	0
2025-07-02	0

---

12. Event with Maximum Sessions

Task: List the event(s) with the highest number of sessions.

SQL Query

```
WITH counts AS (
  SELECT event_id, COUNT(*) AS session_count FROM Sessions GROUP BY event_id
)
SELECT e.event_id, e.title, c.session_count
FROM counts c JOIN Events e ON c.event_id = e.event_id
WHERE c.session_count = (SELECT MAX(session_count) FROM counts);
```

Result

event_id	title	session_count
1	Tech Innovators Meetup	2

---

13. Average Rating per City

Task: Calculate the average feedback rating of events conducted in each city.

SQL Query

```
SELECT e.city, ROUND(AVG(f.rating),2) AS avg_rating, COUNT(f.feedback_id) AS
feedback_count
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
```

```
GROUP BY e.city
ORDER BY avg_rating DESC;
```

Result

city	avg_rating	feedback_count
Chicago	4.5	2
New York	3	1

---

14. Most Registered Events

Task: List top 3 events based on the total number of user registrations.

SQL Query

```
SELECT e.event_id, e.title, COUNT(r.registration_id) AS total_registrations
FROM Events e
LEFT JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_registrations DESC
LIMIT 3;
```

Result

event_id	title	total_registrations
1	Tech Innovators Meetup	2
2	AI & ML Conference	2
3	Frontend Development Bootcamp	1

---

15. Event Session Time Conflict

Task: Identify overlapping sessions within the same event (i.e., session start and end times that conflict).

SQL Query

```
SELECT s1.event_id, s1.session_id AS session1, s2.session_id AS session2,
       s1.title AS title1, s2.title AS title2
FROM Sessions s1
JOIN Sessions s2 ON s1.event_id = s2.event_id AND s1.session_id < s2.session_id
WHERE s1.start_time < s2.end_time AND s2.start_time < s1.end_time;
```

Result

event_id	session1	session2	title1	title2

---

16. Unregistered Active Users

Task: Find users who created an account in the last 30 days but haven't registered for any events.

SQL Query

```
SELECT u.user_id, u.full_name, u.registration_date
FROM Users u
WHERE u.registration_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
  AND u.user_id NOT IN (SELECT user_id FROM Registrations);
```

Result

user_id	full_name	registration_date

---

17. Multi-Session Speakers

Task: Identify speakers who are handling more than one session across all events.

SQL Query

```
SELECT speaker_name, COUNT(*) AS session_count
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(*) > 1;
```

Result

speaker_name	session_count

---

18. Resource Availability Check

Task: List all events that do not have any resources uploaded.

SQL Query

```
SELECT e.event_id, e.title
FROM Events e
LEFT JOIN Resources r ON e.event_id = r.event_id
WHERE r.resource_id IS NULL;
```

Result

event_id	title

---

19. Completed Events with Feedback Summary

Task: For completed events, show total registrations and average feedback rating.

SQL Query

```
SELECT e.event_id, e.title,
  (SELECT COUNT(*) FROM Registrations r WHERE r.event_id = e.event_id) AS
total_registrations,
  ROUND((SELECT AVG(rating) FROM Feedback f WHERE f.event_id = e.event_id),2) AS
avg_rating
```

```
FROM Events e
WHERE e.status = 'completed';
```

Result

event_id	title	total_registrations	avg_rating
2	AI & ML Conference	2	4.5

---

20. User Engagement Index

Task: For each user, calculate how many events they attended and how many feedbacks they submitted.

SQL Query

```
SELECT u.user_id, u.full_name,
  (SELECT COUNT(*) FROM Registrations r WHERE r.user_id = u.user_id) AS events_attended,
  (SELECT COUNT(*) FROM Feedback f WHERE f.user_id = u.user_id) AS feedback_submitted
FROM Users u
ORDER BY u.user_id;
```

Result

user_id	full_name	events_attended	feedback_submitted
1	Alice Johnson	1	0
2	Bob Smith	1	1
3	Charlie Lee	1	1
4	Diana King	1	1
5	Ethan Hunt	1	0

---

21. Top Feedback Providers

Task: List top 5 users who have submitted the most feedback entries.

SQL Query

```
SELECT u.user_id, u.full_name, COUNT(f.feedback_id) AS feedback_count
FROM Users u
JOIN Feedback f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name
ORDER BY feedback_count DESC
LIMIT 5;
```

Result

user_id	full_name	feedback_count
2	Bob Smith	1
3	Charlie Lee	1
4	Diana King	1

---

22. Duplicate Registrations Check

Task: Detect if a user has been registered more than once for the same event.

SQL Query

```
SELECT user_id, event_id, COUNT(*) AS registration_count
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(*) > 1;
```

Result

user_id	event_id	registration_count

---

23. Registration Trends

Task: Show a month-wise registration count trend over the past 12 months.

SQL Query

```
SELECT DATE_FORMAT(registration_date, '%Y-%m') AS reg_month, COUNT(*) AS
registration_count
FROM Registrations
WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY reg_month
ORDER BY reg_month;
```

Result

reg_month	registration_count
2025-04	2
2025-05	2
2025-06	1

---

24. Average Session Duration per Event

Task: Compute the average duration (in minutes) of sessions in each event.

SQL Query

```
SELECT event_id, ROUND(AVG(TIMESTAMPDIFF(MINUTE, start_time, end_time)),2) AS
avg_duration_minutes
FROM Sessions
GROUP BY event_id
ORDER BY event_id;
```

Result

event_id	avg_duration_minutes
1	67.5
2	90
3	120

25. Events Without Sessions

Task: List all events that currently have no sessions scheduled under them.

SQL Query

```
SELECT e.event_id, e.title
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE s.session_id IS NULL;
```

Result

event_id	title